Inter Process Communication

Cooperation:

Two or more independent processes work together by sharing data and/or system resources to achieve a common goal. Where processes are separate by the OS and each of them has their own address.

N.B: sending a copy of the data to each other happens in process communication not in process cooperation.

Problems That we face as a result of cooperation:

Data Consistency Due to Concurrency: refers to the challenge of keeping shared data correct and in a valid state when multiple processes access or modify it concurrently, because concurrent execution (for example through race conditions) can lead to inconsistent or incorrect data.

Racing Conditions: when two or more processes share the same data and they both execute code on it before each of them finish and save the data for the other. leading to invalid data when these processes finally update and write the data.

Deadlocks: Occur when two or more processes each hold a resource and wait indefinitely for other resources held by the other processes, so that none of them can proceed and the program is effectively frozen.

Systems that realizes process cooperation:

Unix System V → the older system Posix System → modern system also linux

Shared Memory:

What is shared memory: Normally each process has it's own memory, for a process to share data with other process it needs to send it via a pipe or a queue message which takes time. Shared memory is mapped inside all the sharing process into their memory address. They all can read and write to it, while still having their own separate memory. This makes data communication very fast.

Coordination is needed: The problem with shared memory of this model is synchronization, if multiple processes read and write to the same memory at the same time race conditions arise, and data inconsistency. This can be solved by mutex and locks (we have multiple other solutions).

Where does shared memory live: Shared memory lives in the memory mapping segment of a process's virtual address space, not literally "RAM" (physical memory), though it ultimately maps to physical RAM.

The memory mapping segment can map the same physical memory into the virtual address space of multiple processes.

Normally, processes cannot access each other's memory, but shared memory is an exception.

This allows direct communication between processes, which is faster than copying data via pipes or message queues.

Shared Memory in Unix:

Unix has a system wide table that stores the shared data information for the entire system.

It includes information like:

The start address: it tells the OS where does the memory chunk of this process starts but in an address way.

The Size: this shared memory segment takes how much space for instance 1kb of shared data for this address.

The owner: who created this shared memory, user, admin.

Privilige

In Unix the shared memory segment does not get erased directly when the process is finished or killed, the shared memory segment and table stays other memory like heap, stack are erased.

The only way to erase this shared memory is if you specifically do so by using a function like unlink() or rebooting the OS.

System Calls with Posix ShMem:

shm_open()	Create a new shared memory segment, or open an existing one. Think of it like **creating or opening a shared "file" in memory**.
ftruncate()	Set the **size** of the shared memory segment. By default, it's 0 bytes, so you must define how big it is.
mmap()	**Attach** the shared memory segment to your process so you can **read/write** it like normal memory.
munmap()	**Detach** the shared memory from your process. You can no longer access it after this.
close()	Close the file descriptor returned by shm_open().
shm_unlink()	**Delete** the shared memory segment from the system.

Message Queue (mq):

A message queue is a kernel-managed queue that allows processes to exchange messages safely.

It uses a linked list data structure to store messages, and fetches them using FIFO style, where each node contains the message data, and some meta data like size priority.

Some message queue allows priority for messages

2 or more processes can have the same messaging queue.

The sending process and the receiving process can both read the queue message

C-System Calls for Message Queues in Posix:

- mq_open(): Create a message queue or access an existing one
- mq_send(): Write (send) a message into a message queue. Blocking operation
- mq_timedsend(): Write (send) a message into a message queue. Blocking operation with a timeout. Normally when a message is being sent to a queue it waits indefinitely if the queue is full, setting a time make it fail after certain time if the queue is full.
- mq_receive(): Read (receive) a message from a message queue. Blocking operation
- mq_timedreceive(): Read (receive) a message from a message queue. Blocking operation with a timeout. Normally when a message is being received from a queue it waits indefinitely if the queue is empty, setting a time make it fail after certain time if the queue is empty.
- mq_getattr(): Request the attributes of a message queue. These are: number of messages in the queue, maximum message size, maximum number of messages
- mq_setattr(): Modify the attributes of a message queue. Like the maximum size
- mq_notify(): Notify the process as soon as a message is available
- mq_close(): Close a message queue. Freeze the queue.
- mq_unlink(): Erase a message queue.

Pipe – Pipeline:

A pipe is a mechanism for inter-process communication (IPC).It allows one process to send data directly to another process. The data flows like a pipe: output of one process → input of another.

Example on how to create a pipe: ls -la | more

The pipe is created using "|" operator, the standard output of " la process" is then used as the standard input for "more process"

Anonymous Pipeline:

A unidirectional pipeline (data goes from x as output to y as input no way it reverses in the middle).

It uses FIFO the data that are outputted first are read first.

We have a buffer (a temporary memory to store data while transmitting) usually 4kb on linux.

The pipeline does not create itself you must create is using pipe().

Write() you write input data to the pipe (buffer) read() just reads it.

The pipeline ceases to exist once the processes are finished execution or have been killed.

When the buffer (pipeline) is filled the writing is blocked.

Is the pipeline is then empty reading is blocked.

Anonymous pipeline requires parent child relationship

Named Pipeline:

FIFO stands for First In First Out → messages are read in the same order they were written.

Unlike anonymous pipes, a named pipe has a name in the filesystem.

Any process that knows the name can write to or read from it.

Used for communication between unrelated processes.

- mkfifo(): creates a named pipe with access rights

`

- open(): opens a named pipe
- read(): reads from a named pipe
- write(): writes to a named pipe
- close(): closes a named pipe
- unlink(): deletes a named pipe

Create pipe	mkfifo("/tmp/myfifo", 0666)	Makes a new named pipe called myfifo with read/write permissions
Open pipe	open("/tmp/myfifo",O_RDONLY/O_WRONLY)	Opens pipe for reading or writing
Write	write(fd, "Hello", 6)	Send data into the pipe
Read	read(fd, buffer, 100)	Receive data from the pipe
Close	close(fd)	Close access to the pipe
Delete	unlink("/tmp/myfifo")	Removes the pipe from the filesystem